

 No description has been provided for this image

Basic Python Part 1

Python is an interpreted language. This means that there is a running Python process that reads and interprets each line of your code, one at a time, and immediately executes that line before moving to the next. This is similar to languages like Matlab and IgorPro, but different from lower level languages like C and Java, in which the program must be compiled and restarted whenever the code changes.

We are interacting with Python through a Jupyter Notebook, one of many interfaces to Python. Jupyter keeps a Python process (called a "kernel") running in the background. Every time you execute a block of code, Jupyter passes your code to the Python interpreter and then displays the result immediately following the code block that was executed.

Execute your first Python command by highlighting the cell below and typing Shift-Return.

```
In [1]: # Python is an interpreted language.  
# It executes commands as it receives them.  
# It ignores lines that begin with '#'. These are called 'comments'.  
  
print("Hello, world")
```

Hello, world

In Python, a single piece of data is called an **object**, and each object has a specific **type** (types are also called "classes"). These include simple types like integers, floats, and strings, and more complex types like lists, arrays, and dictionaries.

In the example above, we created an object with the value "hello, world". If we want to know what type this object has, we can ask the Python interpreter:

```
In [2]: type("Hello, world")
```

```
Out[2]: str
```

hello, world is a string (this type is called `str` in Python).

Note 1: Usually we need to use `print()` if we want output printed in the notebook. However, Jupyter will also display the output of the last line in each cell for us, so it was not necessary to use `print()` in this case.

Note 2: Python allows the use of single or double quotes when defining strings, as long as the same type of quote is used to begin and end each string:

```
In [3]: print('Hello, world 1')
        print("Hello, world 2")
```

```
Hello, world 1
Hello, world 2
```

It is important to know the **types** of your objects, because this will tell you what you can **do** with each object.

In "object-oriented" programming, both data and the functions that operate on that data are encapsulated together in an object. That is, each object gives us access to both its data (an integer, a string, etc.) and a set of functions (called "methods") that we can use to manipulate the data:

```
In [4]: # For example, we can convert a string to all lower-case using its `lower()`
        # We use "dot" notation to access a method from an object:
        "HELLO, WORLD".lower()
```

```
Out[4]: 'hello, world'
```

Exercise 1.1: Capitalize the letters in a string.

'hello' -> 'HELLO'

See: [string methods documentation](#)

```
In [5]: #answer
        'hello'.upper()
```

```
Out[5]: 'HELLO'
```

```
In [6]: # We can also use the `replace()` method to modify a string:
        "Hello, world".replace("Hello", "Howdy")
```

```
Out[6]: 'Howdy, world'
```

Assigning variables

We can assign new variable names to our objects so that we can access them again later:

```
In [7]: # Create a new variable called `hello` and assign it to a string object:
hello = 'Hello, world'

# Now we can access the same object using the variable name:
print(hello)
```

Hello, world

Variable names must conform to two simple rules:

1. May contain only letters, numbers, and underscores
2. May not begin with a number

Note 1: An object in Python has a specific type (int, str, etc.), and this type cannot be changed once the object is created. In contrast, variable names simply point to an object, and have no type of their own. These names can be reassigned to objects of different types at any time.

If you assign a name to an object, then that name can be used in all the same ways as the object itself:

```
In [8]: # We can check the object's type:
print(type(hello))

# We can access the object's methods:
print(hello.replace("Hello", "Hola"))

# We can even assign new names to the same object:
hello2 = hello
print(hello2)
```

```
<class 'str'>
Hola, world
Hello, world
```

A word about names

Python uses a different model than you may be familiar with for assigning names. The assignment operator (=) in Python creates a **reference** to an object in memory. It does **not** create a new variable with the new name. You can test this with the builtin function `id`, which returns a unique identifier for every object.

Now let's play with some of the numerical data types:

```
In [9]: print(type(1.0))
print(type(3))
```

```
<class 'float'>
<class 'int'>
```

There are two basic numerical types here: `float` and `int`. Note that simply writing a decimal point with the number is sufficient to inform Python that a `float` is intended.

Most of the basic arithmetic tools you are familiar with are available for these types:

```
In [10]: a = 1 + 3          # addition / subtraction
        b = 10.0 * 4 / 2    # multiplication / division
        c = 8.5**3         # exponent
        d = 173 % 12        # modulo
        e = 1e2 * (c + a)   # scientific notation, parentheses
        f = 5//2           # integer division
        print(a, b, c, d, e, f)
```

```
4 20.0 614.125 5 61812.5 2
```

Strings also have the sum operation

```
In [11]: hello = 'hello' + ' world'

        print(hello)
```

```
hello world
```

We can also easily convert objects of one type into another. Note that we are not changing the types of each object, but rather creating new objects of different types:

```
In [12]: a = float(3)      # create float from int
        b = int(3.5)       # coerce float to int (this rounds the value down)
        c = float("9.5")   # convert str to float
        d = str(10)        # convert into to str
        print([a, b, c, d])
```

```
[3.0, 3, 9.5, '10']
```

Exercise 1.2: Add two string representations of an integer.

```
'345', '20' -> '365'
```

```
In [13]: #answer
        n1 = '345'
        n2 = '20'

        print("this is wrong:")
        print(n1+n2)
        print
```

```
print("this is correct:")
print(int(n1)+int(n2))
```

this is wrong:

34520

this is correct:

365

Exercise 1.3: Many countries use a comma instead of period to indicate a decimal point. However, Python does not support this notation. Modify the following string and convert it to a float object using tools learned above.

```
strval = "124,78"
```

```
In [14]: strval = "124,78"
floatval = float(strval.replace(',', '.'))
print(floatval, type(floatval))
```

124.78 <class 'float'>

Another very common operation to perform on strings is to split the string into smaller pieces. For example, let's say we want to take the string "Hello, world" and split it into the two parts that come before and after the comma. Strings provide a handy `split()` method for this purpose:

```
In [15]: hello = "Hello, world"
words_from_hello = hello.split(", ")
print(words_from_hello)
```

['Hello', 'world']

If everything is an object, what is `words_from_hello` ?

```
In [16]: type(words_from_hello)
```

Out[16]: list

`list` is a fundamental type in Python.

What is a list? A list is a collection of objects that is:

1. **ordered** - the objects are arranged in a certain order.
2. **addressable** - we can request each object by its address (its position in the list).
3. **mutable** - the list can be changed after it is created.

Note that lists may contain **any** type of Python object, and the objects in a list need not be of the same type.

```
In [17]: # A 'list' is an object that is
```

```
# 1) ordered
# words_from_hello[0] comes before words_from_hello[1]
# There is a method to reverse the order
words_from_hello.reverse()
print(words_from_hello)
words_from_hello.reverse()

# 2) addressable
# Data in a list can be accessed with an index, starting from 0
print(words_from_hello[0])
print(words_from_hello[1])

# 3) mutable
words_from_hello[0] = 'What'      # overwrite the first value
words_from_hello.insert(1, 'wonderful') # insert a new value
words_from_hello.insert(1, 'a')   # insert a new value
print(words_from_hello)
```

```
['world', 'Hello']
Hello
world
['What', 'a', 'wonderful', 'world']
```

Another word about names

Exercise 1.4: The cell below creates the name `my_new_list` and assigns it with `words_from_hello`. Assign a new value to the first element of `my_new_list`. What are the values of `my_new_list` and `words_from_hello`?

```
In [18]: my_new_list = words_from_hello
```

Use the cell below:

```
In [19]: #answer:
print(my_new_list)

my_new_list[0]="It's"
print(my_new_list)
print(words_from_hello)
```

```
['What', 'a', 'wonderful', 'world']
["It's", 'a', 'wonderful', 'world']
["It's", 'a', 'wonderful', 'world']
```

Assignment creates new names, not new objects!

Also, It's important enough to say again:

Python indexes from 0!

```
In [20]: print(words_from_hello[0])
```

It's

```
In [21]: print(words_from_hello[3])
```

world

What just happened? `words_from_hello` is a list of length 3, i.e. it has three elements with indexes 0, 1, and 2. Trying to access an index that would point outside that range (such as 3) produces an error. For now, let's focus on the last line of this error message:

IndexError: list index out of range

There are some important clues in this message:

1. We were accessing a `list` when the error occurred
2. The index we used to access the list (in this case 3) was invalid

Python handles errors by creating an object that describes the error. The type of the error object depends on the kind of error that occurred. Using a bad index here generates an `IndexError`. We will see many other error types later.

```
In [22]: # Common methods for lists
words_from_hello.append('!!')      # add an object to a list
print(words_from_hello)
words_from_hello.remove('wonderful') # remove the first instance of a given object
print(words_from_hello)
words_from_hello.insert(2, 'wonderful') # insert an object at a given index
print(words_from_hello)
words_from_hello.pop(4)             # remove the item at a specific index
print(words_from_hello)
```

```
["It's", 'a', 'wonderful', 'world', '!!']
["It's", 'a', 'world', '!!']
["It's", 'a', 'wonderful', 'world', '!!']
["It's", 'a', 'wonderful', 'world']
```

Exercise 1.5: Using the methods above, change `words_from_hello` such that its value is `['hello','beautiful','world','!']`

```
In [23]: #answer
print(words_from_hello)

words_from_hello[0] = 'hello'
print(words_from_hello)
```

```
words_from_hello.remove('a')
print(words_from_hello)

words_from_hello[1] = 'beautiful'
print(words_from_hello)

words_from_hello.append('!!')
print(words_from_hello)
```

```
["It's", 'a', 'wonderful', 'world']
['hello', 'a', 'wonderful', 'world']
['hello', 'wonderful', 'world']
['hello', 'beautiful', 'world']
['hello', 'beautiful', 'world', '!!']
```

So we have learned that all data in Python is represented as **objects**, that each object is of a specific **type**, and that each type has specific **methods** associated with it (strings have `lower()` and `replace()`, lists have `append()` and `remove()`, etc).

Given an object of a particular type, how can we determine the list of all methods available for that object? We have a few options:

1. Read the documentation for that type.
2. Use Python's built-in help system.
3. Directly list the available attributes of the object.

The documentation for Python's built-in object types can be found [here](#). Later, we will make use of object types that are useful for numerical and scientific work, but that are not built directly into Python. Each of these third-party types will have its own source for documentation.

One very useful feature of Python is that it allows you to embed the documentation for your code directly into the code itself. This allows us to retrieve the documentation directly from the Python interpreter using the `help()` function:

```
In [24]: help(words_from_hello)
```


Help on list object:

```
class list(object)
| list(iterable=(), /)
|
| Built-in mutable sequence.
|
| If no argument is given, the constructor creates a new empty list.
| The argument must be an iterable if specified.
|
| Methods defined here:
|
| __add__(self, value, /)
|     Return self+value.
|
| __contains__(self, key, /)
|     Return key in self.
|
| __delitem__(self, key, /)
|     Delete self[key].
|
| __eq__(self, value, /)
|     Return self==value.
|
| __ge__(self, value, /)
|     Return self>=value.
|
| __getattr__(self, name, /)
|     Return getattr(self, name).
|
| __getitem__(...)
|     x.__getitem__(y) <==> x[y]
|
| __gt__(self, value, /)
|     Return self>value.
|
| __iadd__(self, value, /)
|     Implement self+=value.
|
| __imul__(self, value, /)
|     Implement self*=value.
|
| __init__(self, /, *args, **kwargs)
|     Initialize self.  See help(type(self)) for accurate signature.
|
| __iter__(self, /)
|     Implement iter(self).
|
| __le__(self, value, /)
|     Return self<=value.
|
| __len__(self, /)
|     Return len(self).
|
| __lt__(self, value, /)
|     Return self<value.
```

```
| __mul__(self, value, /)  
|     Return self*value.  
  
| __ne__(self, value, /)  
|     Return self!=value.  
  
| __repr__(self, /)  
|     Return repr(self).  
  
| __reversed__(self, /)  
|     Return a reverse iterator over the list.  
  
| __rmul__(self, value, /)  
|     Return value*self.  
  
| __setitem__(self, key, value, /)  
|     Set self[key] to value.  
  
| __sizeof__(self, /)  
|     Return the size of the list in memory, in bytes.  
  
| append(self, object, /)  
|     Append object to the end of the list.  
  
| clear(self, /)  
|     Remove all items from list.  
  
| copy(self, /)  
|     Return a shallow copy of the list.  
  
| count(self, value, /)  
|     Return number of occurrences of value.  
  
| extend(self, iterable, /)  
|     Extend list by appending elements from the iterable.  
  
| index(self, value, start=0, stop=9223372036854775807, /)  
|     Return first index of value.  
  
|     Raises ValueError if the value is not present.  
  
| insert(self, index, object, /)  
|     Insert object before index.  
  
| pop(self, index=-1, /)  
|     Remove and return item at index (default last).  
  
|     Raises IndexError if list is empty or index is out of range.  
  
| remove(self, value, /)  
|     Remove first occurrence of value.  
  
|     Raises ValueError if the value is not present.  
  
| reverse(self, /)
```

```

|         Reverse *IN PLACE*.
|
|     sort(self, /, *, key=None, reverse=False)
|         Sort the list in ascending order and return None.
|
|         The sort is in-place (i.e. the list itself is modified) and stable
(i.e. the
|         order of two equal elements is maintained).
|
|         If a key function is given, apply it once to each list item and sort
them,
|         ascending or descending, according to their function values.
|
|         The reverse flag can be set to sort in descending order.
|
| -----
| Class methods defined here:
|
|     __class_getitem__(...) from builtins.type
|         See PEP 585
|
| -----
| Static methods defined here:
|
|     __new__(*args, **kwargs) from builtins.type
|         Create and return a new object.  See help(type) for accurate signatu
re.
|
| -----
| Data and other attributes defined here:
|
|     __hash__ = None

```

In the help documentation above, we see many methods that begin with an underscore `_`. These methods are considered private to the object; ignore these for now. The remaining methods are meant to be public (we are encouraged to use them).

Another very useful feature of Python is that it allows **introspection**--we can ask the Python interpreter questions like "what variables are available from here", "what type is this object", and "what methods are available through this object".

```

In [25]: # List all attributes of `words_from_hello`.
print(dir(words_from_hello))

```

```

['_add_', '_class_', '_class_getitem_', '_contains_', '_delattr_',
'_delitem_', '_dir_', '_doc_', '_eq_', '_format_', '_ge_', '_ge
tattribute_', '_getitem_', '_gt_', '_hash_', '_iadd_', '_imul_',
'_init_', '_init_subclass_', '_iter_', '_le_', '_len_', '_lt_',
'_mul_', '_ne_', '_new_', '_reduce_', '_reduce_ex_', '_repr_',
'_reversed_', '_rmul_', '_setattr_', '_setitem_', '_sizeof_', '_s
tr_', '_subclasshook_', 'append', 'clear', 'copy', 'count', 'extend', 'in
dex', 'insert', 'pop', 'remove', 'reverse', 'sort']

```

Exercise 1.6: Sort words in `words_from_hello` alphabetically.

```
In [26]: #answer
words_from_hello.sort()
print(words_from_hello)
```

```
['!', 'beautiful', 'hello', 'world']
```

You can create new lists using a comma-separated list of objects inside square brackets `[]`. Lists may contain *any* kind of python object, including, for example, other lists. Every element in a list need not be of the same type.

```
In [27]: my_new_list = [hello, 'hey', 1.0, 3, words_from_hello]
print(my_new_list)
```

```
['Hello, world', 'hey', 1.0, 3, ['!', 'beautiful', 'hello', 'world']]
```

Python has a type called `range` that describes sequences of `int` s. We can create `range` s by calling the builtin function also called `range`. Casting these to a `list` is an easy way to create lists of integers.

Notice that the entries here run from 0 to 9 when we ask for a list of length 10.

```
In [28]: my_ints = list(range(10))
print(my_ints)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

`len()` is a convenient function to find out the length of certain types of objects, like strings and lists:

```
In [29]: print(len(my_ints))
print(len(words_from_hello))
print(len(hello))
```

```
10
```

```
4
```

```
12
```

We have already seen that we can access specific items in a list by indexing:

```
second_item = my_list[1]
```

We can also access a subset of a list with **slicing**:

```
In [30]: print(my_new_list)
print(my_new_list[2:4])    # The upper limit of '4' is non-inclusive; this w
```

```
print(my_new_list[:3])      # This will return the first three elements
print(my_new_list[-2:])     # You can also index from the end of an array. 7
```

```
['Hello, world', 'hey', 1.0, 3, ['!', 'beautiful', 'hello', 'world']]
[1.0, 3]
['Hello, world', 'hey', 1.0]
[3, ['!', 'beautiful', 'hello', 'world']]
```

Exercise 1.7: Replace the suffix in a filename string using slice notation (and thus without using the `replace` method).

'my_file.txt' -> 'my_file.pdf'

```
In [31]: #answer
filename = 'myfile.txt'
filename = filename[:-3]+'pdf'
print(filename)
```

myfile.pdf

Tuples are another type of object, very similar to a list.

A tuple is 1) **addressable**, 2) **ordered**, 3) **immutable**.

The primary difference from a 'list' is that tuples are **immutable**. They cannot be altered once created. They are indicated by parentheses rather than brackets.

```
In [32]: my_new_tuple = ('hey', 2.0, 3, words_from_hello)
```

```
In [33]: my_new_tuple[0] = 'hi'
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[33], line 1
----> 1 my_new_tuple[0] = 'hi'

TypeError: 'tuple' object does not support item assignment
```

In trying to change the first element of the tuple `my_new_tuple`, we get an error called `TypeError`.

A word of caution: If the elements in the tuple are themselves mutable (such as lists), then you can still change the data within those elements.

```
In [34]: print(my_new_tuple)
my_new_tuple[3].remove("hello") # removing an element of a list. The list i
my_new_tuple[3][0] = 'hello'    # assigning the first element of a list. The
print(my_new_tuple)
```

```
('hey', 2.0, 3, ['!', 'beautiful', 'hello', 'world'])
('hey', 2.0, 3, ['hello', 'beautiful', 'world'])
```

Fun with tuples!

Tuples and lists allow you to perform "multiple assignment"

```
In [35]: x,y,z = 1,2,3
print(x)
print(y)
print(z)
```

```
1
2
3
```

```
In [36]: x,y,z,w = my_new_tuple
print(x)
print(y)
print(z)
print(w)
```

```
hey
2.0
3
['hello', 'beautiful', 'world']
```

Lists and tuples have sum operations. Note: These operations create **new** objects.

```
In [37]: my_list = ['why'] + words_from_hello
my_tuple = (1,) + (2,3)

print(my_list)
print(my_tuple)
```

```
['why', 'hello', 'beautiful', 'world']
(1, 2, 3)
```

In certain cases, you can cast data types.

We saw above that we can access items from lists and tuples by indexing with integers:

```
an_item = a_list[7]
```

This is a very useful structure if you already happen to know which index you are looking for. There are many scenarios, however, where this assumption is not true. For example, consider a phonebook (if you can remember what a phonebook looks like). We know the name of a person we want to call, and with that information we can retrieve their phone number. If the phone numbers were stored in a list, then we would need to know that "Michael" is the 7354th entry in the list in order to retrieve his phone number. Not so helpful.

What we would like is an object like a list that lets us retrieve its items by **name** rather than by **position**:

```
phone_number = phonebook['Michael']
```

This turns out to be a very common problem in programming, and Python solves it by providing a built-in type called a `dict` (short for dictionary). Other languages call this a "mapping", "hash map", or sometimes just "object".

A `dict` maps arbitrary (mostly) objects to elements. A `dict` is a data structure that is 1) **addressable**, 2) **unordered**, 3) **mutable**

```
In [38]: # A phonebook, the awkward way:
phonebook_names = ['Luke', 'Michael', 'Shawn']
phonebook_numbers = [9194280943, 2048675309, 5780239432]

# Look up Luke's index, and then phone number:
luke_index = phonebook_names.index('Luke')
print(phonebook_numbers[luke_index])

# A phonebook, as Guido intended:
phonebook = {'Luke': 9194280943, 'Michael': 2048675309, 'Shawn': 5780239432}
print(phonebook['Luke'])
```

```
9194280943
9194280943
```

Empty dictionaries are created with `{}`. You can add to a `dict` by assignment to a new key. Trying to access a key that isn't there without assignment generates a `KeyError`.

```
In [39]: words_dict = {}
print(words_dict)

words_dict['X'] = 1
words_dict['Y'] = 2
words_dict['Z'] = "hello"

print(words_dict)

words_dict['Z']

{}
{'X': 1, 'Y': 2, 'Z': 'hello'}
```

```
Out[39]: 'hello'
```

Exercise 1.8: Make a dictionary with 'school', 'major', 'hobby', and 'year of graduation' as keys and provide values appropriate for yourself.

```
In [40]: #answer
my_dictionary = {}
my_dictionary['school'] = 'hard knocks'
my_dictionary['major'] = 'PE'
my_dictionary['hobby'] = 'neuroscience'
my_dictionary['year of graduation'] = '1984'
print(my_dictionary)
```

```
{'school': 'hard knocks', 'major': 'PE', 'hobby': 'neuroscience', 'year of graduation': '1984'}
```

Exercise 1.9: Make a dictionary where the keys are your name and those of the two people sitting closest to you. The values should be the dictionaries as in the previous exercise.

```
In [41]: #answer
our_dictionary = {}
our_dictionary['me'] = my_dictionary
our_dictionary['my neighbor'] = {}
print(our_dictionary)
```

```
{'me': {'school': 'hard knocks', 'major': 'PE', 'hobby': 'neuroscience', 'year of graduation': '1984'}, 'my neighbor': {}}
```

Review

1. All variables are objects. Objects have methods.
2. Data Structures: list, tuple, dict
3. Data Types: str, float, int

Exercises

Exercise 1.10: The following variables are allowed (T/F):

1. 3Dgraph
2. hello_world
3. January2008
4. Trial#1
5. myData

1. 3Dgraph - **False**, variables cannot begin with a digit
2. hello_world - **True**

3. January2008 - **True**
4. Trial#1 - **False**, variables must only use letters, digits or underscores
5. myData - **True**

Exercise 1.11: Write a series of statements that perform the following operations:

1. Assign 3 to 'a'.
2. Add 5 to 'a' and assign the result to 'b'.
3. Divide 'b' by 2 and assign the result to 'c'.
4. Print 'c'.

```
In [42]: #answer
a = 3
b = a + 5
c = b / 2
print(c)
```

4.0

Exercise 1.12: Convert a temperature in Celsius to Fahrenheit ($f = 9/5 \times c + 32$) (Hint: keep in mind how using float and int types can affect multiplication and division).

```
In [43]: #answer
c = 38
f = (9.0/5.0)*c + 32
print(f)
```

100.4

Exercise 1.13: Print the last five objects in this list: [1,2,3,4,5,6,7,8,9]

```
In [44]: #answer
list = [1,2,3,4,5,6,7,8,9]
print(list[4:])
```

[5, 6, 7, 8, 9]

Exercise 1.14: Replace every instance of a character (e.g. 'l') in a string with another (e.g. 'y').

'hello' -> 'heyho'

```
In [45]: #answer
s = 'hello'
s = s.replace('l','y')
print(s)
```

heyyo

Exercise 1.15: Reverse the word order in a sentence.

```
In [46]: sentence = 'that rug really tied the room together'
```

```
In [47]: #answer
sentence_list = sentence.split(" ")
sentence_list.reverse()

reversed_sentence = " ".join(sentence_list)
print(reversed_sentence)
```

together room the tied really rug that

Exercise 1.16: Write a function to get the the decimal component of a float without using subtraction.

`12.345` -> `0.345`

```
In [48]: #answer
def get_decimal(in_val):
    new_val = str(float(in_val)) #cast to float first, just in case it wasn't
    deci = new_val.split(".")[1] #split on the decimal
    return float('0.'+deci) #add a '0.' to the front of the string, cast as float

get_decimal(3.14159265359)
```

Out[48]: 0.14159265359

Exercise 1.17: Find the maximum and minimum element in a list of numbers. (This can be done with tools you have just seen or with a built in function.)

```
In [49]: vals = [8,1,95,3,-1,55,4.6]
print(vals)

#using methods from above:
vals.sort()
print('using list sorting:')
print('minimum:')
```

```

print(vals[0])
print
print('maximum:')
print(vals[-1])
print

#using built ins:
print('using min and max built-in functions:')
print('minimum:')
print(min(vals))
print
print('maximum:')
print(max(vals))

```

```

[8, 1, 95, 3, -1, 55, 4.6]
using list sorting:
minimum:
-1
maximum:
95
using min and max built-in functions:
minimum:
-1
maximum:
95

```

Exercise 1.18: Take a look at the methods available on the `dict` object. Using the supplied dictionaries below, insert the contents of dictionary `b` into dictionary `a`.

```

In [50]: a = { 'a': 100, 'c': 400 }
          b = { 'a': 300, 'b': 200 }
          a.update(b)
          print(a)

{'a': 300, 'c': 400, 'b': 200}

```